**European Shelter Dog Adoption Platform**

# Brief Description of the System

The platform aims to create **Europe's first centralized database and web page** for all shelter dogs across Europe. The goal is to improve adoption rates, reduce shelter overcrowding, and provide an accessible, multilingual adoption platform for potential adopters and animal shelters.

# Main System Functions

- **Search Engine for Dogs**: Users can search for dogs based on filters like breed, age, size, sex, and location.
- **Dog Profiles**: Each dog will have a dedicated profile page with medical details, personality traits, and contact information.
- **Contact Form on Each Dog's Page**: Users can send messages or questions about a dog via a contact form that sends emails to the shelter.
- **Shelter Management System**: Shelters can register and manage their listed dogs, update profiles, and review inquiries.
- **Multilingual Support**: The platform will support multiple languages to cater to different European countries.
- **Automated Processes**: To minimize administrative workload, automations will be in place for profile updates, notifications, and translations.

# Technologies

- **Frontend:** Next.js (React)
- **Backend:** Django + Django REST Framework (DRF)
- **Database:** PostgreSQL (relational database for scalability)
- **Hosting:** Vercel (for frontend), AWS/DigitalOcean (for backend)
- **Authentication:** Only shelters need to register and log in; general users can freely browse the site.

# Basic Entities (Proposal)

## Shelter

- **ID** (Primary Key)
- **Name**
- **Location** (Country, City, Address)
- **Contact Information** (Email, Phone, Website)
- **Verified Status** (Yes/No)

## Dog

- **ID** (Primary Key)
- **Name**
- **Age**
- **Breed**
- **Size** (Small, Medium, Large)
- **Sex** (Male/Female)
- **Medical History**
- **Personality Traits**
- **Adoption Status** (Available, Adopted, Pending)
- **Shelter** (Foreign Key)
- **Photos** (Uploaded images)
- **Contact Form** (Allows users to send messages or questions via email)

## Contact Form Inquiry

- **ID** (Primary Key)
- **Dog** (Foreign Key)
- **User Name** (optional field)
- **User Email**
- **Message**
- **Submission Date**

# Functionalities

## Search & Filters

- Users can filter dogs by **breed, age, size, sex, and country**.
- Search results should prioritize **location-based matches**.

## User Access

- Users can browse the platform **freely without registration**.
- Each dog page includes a **contact form** for direct inquiries via email.

## Shelter Dashboard

- Shelters can **add, edit, and remove** dog profiles.
- View and manage **adoption inquiries**.

## Automated Processes

- **Profile Expiry Notifications**: Shelters get alerts for outdated profiles.
- **Translation Services**: AI-powered translation for dog descriptions.
- **Adoption Status Updates**: Automatic updates when a dog is adopted.

# Additional Requirements

- **Aesthetic and Functional UI**: Clean, accessible design optimized for different devices.
- **Data Validation**: User inputs must be validated to prevent incorrect or incomplete data entries.
- **Scalability**: The system should support **thousands of shelters and listings** without performance issues.
- **Security Measures**: Protect shelter data with encryption and authentication protocols.

# Potential Future Expansions

- **Mobile App**: Develop a mobile version for broader reach.
- **AI Matching**: Implement AI-powered dog-adopter matching based on personality and lifestyle.
- **Donation System**: Allow users to donate to shelters directly via the platform.
- **Volunteer & Fostering Options**: Extend the system to facilitate temporary fostering and volunteering opportunities.

---

This document serves as an initial roadmap for the development of the **European Shelter Dog Adoption Platform**. Further refinements will be made based on user feedback and testing.

**Development workflow**. Here's a structured approach:

## 1. Set Up Repositories and Project Structure

- Create a **GitHub/GitLab repository** for version control.
- Define the **project structure** for both frontend (Next.js) and backend (Django).
- Use a **monorepo** (both frontend & backend in one repo) or separate repos for better organization.

## 2. Backend (Django + DRF) Setup

- Set up a **Django project** and configure **PostgreSQL**.
- Install **Django REST Framework** for API development.
- Define the **models** (Dog, Shelter, Contact Form Inquiry).
- Implement **basic API endpoints** for shelters and dogs.
- Set up **JWT authentication** (only for shelters).
- Write initial **unit tests** for key features.

## 3. Frontend (Next.js) Setup

- Create a **Next.js project** and set up routing.
- Choose a **UI library** (ShadCN, Tailwind CSS) for styling.
- Implement basic pages like **home, search, and dog profile pages**.
- Set up **API calls** to fetch dog data from the backend.

## 4. Database and Hosting Preparation

- Set up a **PostgreSQL database** (local for development, cloud for production).
- Choose a **cloud provider** (AWS/DigitalOcean) for backend hosting.
- Set up **Vercel** for hosting the frontend.

## 5. DevOps and CI/CD Setup

- Configure **Docker** (optional) for development consistency.
- Set up **CI/CD pipelines** to automate testing & deployment.
- Establish **staging environments** for testing before going live.

## 6. Development Roadmap

After setting up the basics, start with:
 **MVP Features**: Search engine, dog profiles, contact form, shelter management.
 **Testing & Feedback**: Get early feedback from shelter partners.
 **Multilingual Support & Automations**: Implement language translations and process automation.